

FPGA Experiment Report

Experiment 5 — PS/2 Keyboard Interface

Task: FPGA 5 — Keyboard (PS/2 interface)

Student 1: Mahmood Stitia

ID: 214032682

Student 2: Saleh Khalil

ID: 213310485

Part 1 — Status update

The table below lists the submitted files and their status. The status is one of *Done* (module and simulation written and passing), *Partially done* (started but not fully working — explained in the notes), or *Not started*.

File / Step	Status	Notes / comments
Ps2_Interface.v	Done	PS/2 receiver; testbench passes
Ps2_Interface_tb.v	Done	Self-checking; all scenarios pass
Ps2_Display.v	Done	7-seg + LED strobe (CDC to 100 MHz)
Ps2_Top.v	Done	Top level, FPGA pin wiring
task1.xdc	Done	Pin + clock constraints

Issues

Any errors present at the time of submission are described here (per module / simulation, with what the error means).

- Ps2_Interface.v:
- Ps2_Interface_tb.v:
- Ps2_Display.v:
- Ps2_Top.v:
- task1.xdc:

Module descriptions

Ps2_Interface

The Ps2_Interface module receives the scan-codes coming from the PS/2 keyboard and produces two outputs: `scancode` and `keyPressed`. It follows the PS/2 serial protocol and detects both key presses and key releases.

To capture the serial data driven by the keyboard we use a shift register that collects the 11-bit PS/2 frame (start bit, 8 data bits, parity, stop bit). Each bit is shifted in on the *falling edge* of PS2Clk, since that is where the keyboard guarantees the data is stable. Because PS2Clk is not

free-running and stops right after the stop bit, the whole module is clocked by `PS2Clk` and is reset by an *active-low asynchronous* reset `rstn`, so it can be cleared even while the keyboard clock is idle.

A 4-bit counter (`bitcount`) tracks how many bits of the current frame have arrived; when it reaches the stop-bit position (`frame_done`) the byte is evaluated — we never wait for an edge *after* the stop bit, because the keyboard does not produce one. We keep a 22-bit shift register so the *current* byte and the *previous* byte are both visible at once; this is what lets us recognise the two-frame `0xE0` (extended key) and `0xF0` (release) sequences. On a complete frame we decide whether to drive `scancode` according to the case: a normal make-code is shown and pulses `keyPressed`; `0xE0/0xF0` prefixes are ignored (no display, no pulse); a make-code following `0xF0` is treated as a release. To emit `keyPressed` only *once* on a genuine new press we keep `held_code` (the code of the key currently held) and `key_down` (whether a key is considered pressed), so typematic auto-repeats of the same key do not generate extra pulses, and a re-press after a release pulses again.

Ps2_Interface_tb

The `Ps2_Interface_tb` module verifies `Ps2_Interface`. Since PS/2 communication is much slower ($\approx 20\text{--}30\text{ kHz}$) than the FPGA system clock (100 MHz), the bench manually drives `PS2Clk` and `PS2Data`, sending the bits one by one at realistic intervals. It uses reusable tasks (`ps2_bit`, `send_byte`) to send framed bytes easily — start, 8 data LSB-first, odd parity, stop — and idles the clock high between frames. The simulation is self-checking and prints `Test Passed - Ps2_Interface_tb` when every scenario passes; it confirms that the interface captures the serial data, identifies key presses, and produces the expected `scancode` and `keyPressed` before we move to hardware. The waveforms are shown in *Part 2*.

Ps2_Display

The `Ps2_Display` module receives the scan-code of the pressed key from `Ps2_Interface` and shows it on the two right-hand 7-segment digits. It splits the byte into two hex nibbles and time-multiplexes them with a clock divider (the two left digits are blanked). The displayed value latches until the next key press. To tell D from 0 and B from 8 we use lowercase `d/b` glyphs in the decode table (the dot `dp` could be lit instead). When a key press is detected a user `led` is pulsed with a short, eye-visible strobe produced by an internal countdown timer. Because `scancode/keyPressed` arrive from the slow `PS2Clk` domain, this module first passes `keyPressed` through a 3-FF synchroniser, edge-detects it, and samples `scancode` on that settled pulse — a safe clock-domain-crossing handshake into the 100 MHz domain.

Ps2_Top

The `Ps2_Top` module integrates the keyboard interface with the display. It connects `Ps2_Interface`, which reads scan-codes from the keyboard, to `Ps2_Display`, which shows the scan-code on the 7-segment display and drives the LED. It routes the signals between the two modules and handles the clock and reset inputs: the board push-button `btnC/reset` is active-high, so it is inverted to the active-low `rstn` used by both sub-modules (an XDC cannot invert a pin). This is the top-level module used to implement the complete design on the FPGA.

Part 2 — Simulations and answers

This part contains the simulation of `Ps2_Interface` (annotated waveforms) followed by the answers to the lab's questions. (*Per the PDF, only `Ps2_Interface.v` is simulated.*)

Simulation — Ps2_Interface

[paste screenshot here]

Annotated waveform of *Ps2_Interface_tb*: *PS2Clk*, *rstn*, *PS2Data*, *scancode[7:0]*, *keyPressed*. Draw arrows on: the reset test, the first make-code, the typematic repeats, the extended key, and the release / re-press.

What the waveform shows (annotate the real screenshot with arrows):

1. **Reset test** — with *rstn*=0 the outputs are cleared even while *PS2Clk* is idle (the async-reset advantage).
2. **First make-code** 0x2E — after the stop-bit edge *scancode* becomes 2E and *keyPressed* pulses exactly once.
3. **Typematic repeats of** 0x2E — the same code is sent twice more; *scancode* stays 2E and *keyPressed* does *not* pulse again.
4. **Extended key** 0xE0, 0x5A — the 0xE0 frame leaves *scancode* unchanged and produces no pulse; the trailing 0x5A sets *scancode*=5A with one pulse.
5. **Release** 0xF0, 0x5A — neither frame pulses; the displayed code is held.
6. **Re-press** 0x5A **after release** — pulses again (a new press after a release is a new event).

[paste screenshot here]

Zoom-in showing that the data is sampled on the negative edge of *PS2Clk* (arrow on a falling edge where *scancode* / *keyPressed* react).

Answer — Asynchronous reset (advantages & disadvantages)

A flip-flop can be reset *synchronously* (the reset is sampled only on the active clock edge) or *asynchronously* (the reset appears in the sensitivity list and acts the instant it is asserted):

always @(posedge clk)	synchronous: reset acts only on a clock edge
always @(posedge clk or negedge rstn)	asynchronous: reset acts immediately

	Asynchronous reset	Synchronous reset
Advantage	Resets the logic <i>immediately</i> , even when <i>no clock edge is available</i> . Critical in this lab because PS2C1k is idle most of the time.	Easy static-timing analysis; the reset is “just another data signal”, no special routing, no recovery / removal checks.
Disadvantage	Reset <i>removal</i> (de-assertion) is dangerous: releasing it too close to a clock edge violates recovery / removal time and can cause metastability ; needs a reset synchroniser and dedicated async-reset routing.	Needs a toggling clock to take effect — useless while the clock is idle; the reset pulse must be at least one clock period wide to be captured.

Why it matters here. The keyboard clock PS2C1k toggles only during a transmission and is idle (high) otherwise. A synchronous reset on that domain would not act while the clock is idle, so Ps2_Interface uses an **active-low asynchronous reset** rstn (see Figure 1 of the assignment: the push-button reset is inverted to ~reset/rstn). The de-assertion hazard is handled on the fast side by a multi-FF synchroniser (see Ps2_Display).

Answer — PS/2 interface (frame, non-toggling clock, key-press detection)

The PS/2 keyboard is a device-driven serial interface on two wires: PS2C1k (clock generated by the keyboard, $\approx 10\text{--}17\text{ kHz}$) and PS2Data. One byte is sent as an **11-bit frame, LSB first**, and the host samples PS2Data **on the falling edge of PS2C1k**:

Bit 0	Bits 1–8	Bit 9	Bit 10
Start (0)	Data D0..D7 (LSB first)	Parity (odd)	Stop (1)

Key facts used by the design:

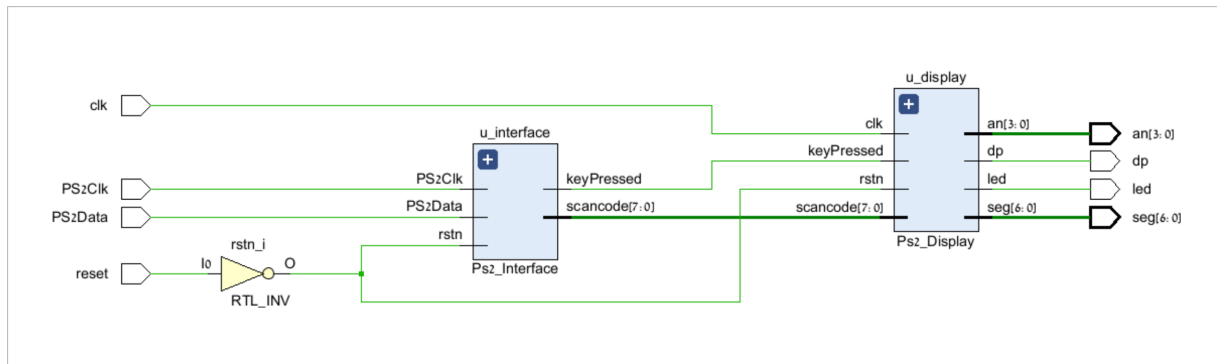
- **Make code** — sent on key press (e.g. keypad 5 $\rightarrow 0x2E$).
- **Typematic repeat** — while a key is held, the make code is re-sent repeatedly; keyPressed must pulse *only once* (on the first press).
- **Break code** — on release: $0xF0$ followed by the make code.
- **Extended keys** — prefixed with $0xE0$; the $0xE0$ must be ignored and only the trailing byte displayed.

Non-toggling clock $\Rightarrow$ decode early. Because PS2C1k stops after the stop bit, the module must *not* wait for an edge after the frame. Ps2_Interface decodes the byte *on* the stop-bit edge (the last edge the keyboard actually produces), using a 22-bit shift register that holds the current frame and the previous one so two-frame sequences ($0xE0/0xF0$) can be recognised.

Two clock domains (CDC). scancode/keyPressed are produced in the slow PS2C1k domain and consumed in the fast 100 MHz clk domain of Ps2_Display. keyPressed is passed through a 3-FF synchroniser and edge-detected; scancode is captured on that already-settled pulse — a simple, safe data + valid CDC handshake. This is also why the logic analyser cannot debug both domains in a single ILA core.

Elaborated Design

Top module



RTL-elaborated schematic of *Ps2_Top*: *u_interface* (*Ps2_Interface*, *PS2Clk* domain) feeds *keyPressed* and *scancode[7:0]* into *u_display* (*Ps2_Display*, 100 MHz domain); the *reset* push-button passes through an *RTL_INV* (*rstn_i*) into *rstn*. Inputs *clk*, *PS2Clk*, *PS2Data*, *reset* are on the left and outputs *an[3:0]*, *dp*, *led*, *seg[6:0]* on the right.

Ps2_Interface

[paste screenshot here]

RTL-elaborated schematic of *Ps2_Interface*: the 22-bit shift register, the *bitcount* counter, the *held_code* / *key_down* registers and the make / break / extend decode logic clocked by *PS2Clk*.

Ps2_Display

[paste screenshot here]

RTL-elaborated schematic of *Ps2_Display*: the 3-FF *keyPressed* synchroniser, the *scancode* capture register, the LED strobe down-counter, the refresh clock divider and the hex→7-seg ROM (*Seg_7_Display*).

Implementation

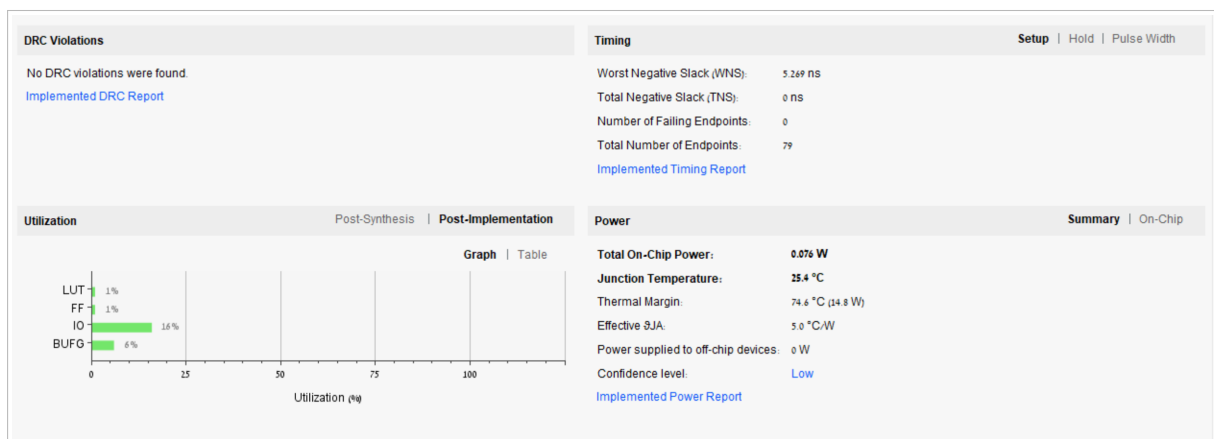
Implementation schematic

[paste screenshot here]

Post-implementation schematic: every external port now goes through an *IBUF/obuf*, the two clocks (*clk* and *PS2clk*) through *BUFG* global buffers, and the logic mapped onto LUTs and flip-flops inside *ps2_interface* and *Ps2_Display*.

Elaborated vs. implemented. The elaborated view shows the behavioural structure as written in Verilog (generic RTL primitives, abstract buses). After synthesis / implementation Vivado maps the design onto real FPGA primitives — LUTs, flip-flops with their async reset, *IBUF/obuf* I/O buffers and *BUFG* global clock buffers — and applies optimisations (logic merging, removing unused paths). The two *create_clock* constraints (100 MHz *clk* and the slow *PS2clk*) are declared *asynchronous* to each other (*set_clock_groups*), so no real path is timed across the two domains.

Project summary



Post-implementation project summary. No DRC violations; timing met ($WNS = 5.269\text{ ns}$, 0 failing endpoints); utilization is small (LUT & FF $\approx 1\%$, IO 16%, BUFG 6%); total on-chip power is 0.076 W with a junction temperature of 25.4°C.

Timing summary

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS): 5.269 ns		Worst Hold Slack (WHS): 0.330 ns		Worst Pulse Width Slack (WPWS): 4.500 ns	
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns		Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	
Total Number of Endpoints: 79		Total Number of Endpoints: 79		Total Number of Endpoints: 57	
All user specified timing constraints are met.					

Design Timing Summary after implementation. The Setup ($WNS = 5.269\text{ ns}$), Hold ($WHS = 0.330\text{ ns}$) and Pulse-Width ($WPWS = 4.500\text{ ns}$) slacks are all positive with 0 failing endpoints — “All user specified timing

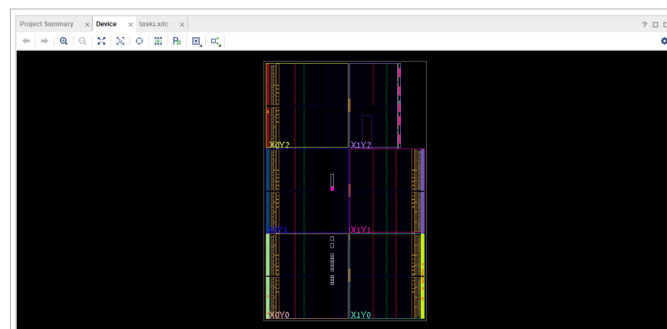
constraints are met”.

Critical Path

[paste screenshot here]

Intra-Clock Paths report: the worst (Path 1) path with its slack, number of logic levels, and total / logic / net delay against the 10 ns requirement (circle the Total Delay).

Device



Device (floorplan) view of the placed-and-routed design on the Artix-7 (XC7A35T). The few used slices are highlighted across the clock regions (X0Y0–X1Y2); the design occupies only a tiny fraction of the fabric.

Problems we faced

On hardware the design behaves as specified: pressing a numeric-keypad key shows its 8-bit make-code as two hex symbols on the two right-hand 7-segment digits (e.g. 5 → 2E); each new press blinks the LED once with a short strobe (holding the key does *not* re-blink); the displayed code latches until the next press; and extended keys show only their last two hex symbols (the 0xE0 prefix is invisible).

(Describe the problems you actually hit in the lab and how you solved them. For example:)

- *(fill in)* where to put the 0xF0/0xE0 / two-key handling logic — in the interface vs. the display module — and the wrong display behaviour seen before moving it all into Ps2_Interface.
- *(fill in)* the keyboard clock being idle most of the time (decode before the stop bit, async reset).
- *(fill in)* CDC glitches on keyPressed and debugging the two clock domains in separate ILA cores.

[paste screenshot here]

Board photo: keypad plugged into the USB host port, a key pressed and its hex code shown on the two right digits with the LED lit.

Appendix — Verilog Source Code

The full source of the submitted files is listed below. Paste the code of each file inside its block.

Ps2_Interface.v

```
1 // --- paste Ps2_Interface.v here ---
```


Ps2_Interface_tb.v

```
1 // --- paste Ps2_Interface_tb.v here ---
```

Ps2_Display.v

```
1 // --- paste Ps2_Display.v here ---
```

Seg_7_Display.v

```
1 // --- paste Seg_7_Display.v here ---
```

Ps2_Top.v

```
1 // --- paste Ps2_Top.v here ---
```

task1.xdc

```
1 ## --- paste task1.xdc here ---
```